Solving the 1D Buckley–Leverett problem with AMGCL

1D Buckley–Leverett notes

Abstract

This note describes the sequential implicit program in `src/buckley_leverett.cpp`. Saturation is advanced by implicit upwind Newton iterations. Pressure is a two-point flux discretization of the total-velocity equation. Both linear systems are solved by AMGCL 1.4.3. The derivation of $f(S)$, the Welge tangent, and the bidiagonal Jacobian is in `theory.tex`.

1 What is integrated

The unknowns are the cell water saturation S_i and the cell pressure p_i on $x \in [0, 1]$. Time is in pore volumes injected. The equations are

$$\frac{\partial S}{\partial t} + \frac{\partial f(S)}{\partial x} = 0, \qquad \frac{d}{dx}\left(k(x)\, \lambda_t(S) \frac{dp}{dx} \right) = 0, \tag{1}$$

with the rate boundary condition and the outlet pressure

$$-k\lambda_t \frac{dp}{dx} = 1, \qquad p(1) = 0. \tag{2}$$

Corey data are $S_{wc} = S_{or} = 0.2$, $n_w = n_o = 2$, $k^0_{rw} = k^0_{ro} = 1$, $\mu_w = 1$, $\mu_o = 2$, so the endpoint mobility ratio is $M = 2$. Absolute permeability is dimensionless and piecewise constant:

$$k(x) = \begin{cases} 0.1, & 0.45 \le x < 0.55, \\ 1, & \text{otherwise.} \end{cases} \tag{3}$$

The streak steepens dp/dx. It does not change S. In one dimension, incompressible flow with only an inlet and an outlet has $\partial u_t / \partial x = 0$, and the inlet rate fixes $u_t = 1$. Fractional flow does not depend on k.

The initial saturation is S_{wc} in every cell. Cell centers are $x_i = (i + \frac{1}{2})\Delta x$.

2 One time step

The target step is limited by the shock CFL,

$$\Delta t = \mathrm{CFL}\, \frac{\Delta x}{v_s}, \tag{4}$$

with default CFL = 1. The default final time is $t = 0.55/v_s$, which places the Welge shock at $x = 0.55$. Both defaults can be overridden by

```
buckley [cells] [pore_volumes] [cfl]
```

The loop is sequential, sometimes called sequential implicit for the saturation part:

1. Given S^n, solve the nonlinear saturation step for S^{n+1} at the fixed total velocity 1. Newton's matrix changes at every iteration and is solved with BiCGStab and algebraic multigrid.

2. Given S^{n+1}, assemble and solve the linear pressure system with CG and algebraic multigrid. The pressure is a diagnostic in this configuration: the saturation update does not read it, because the rate is already known. The solve is still checked, because the discrete face fluxes must reproduce the boundary rate.

3. Advance time and add the outlet water volume $f(S_{N-1}^{n+1})\,\Delta t$.

If Newton or the linear solver fails, Δt is halved, up to twelve times, and the step is repeated from S^n. The default 800-cell run does not cut the step.

3 Saturation: Newton on the upwind residual

The residual and its Jacobian are derived in `theory.tex`. In code they are `residual_linf` and `assemble_transport`. With $a = \Delta x/\Delta t$ and with the inlet flux equal to 1,

$$R_i = a\,(S_i - S_i^n) + f(S_i) - f_{\mathrm{up},i}, \qquad f_{\mathrm{up},0} = 1, \quad f_{\mathrm{up},i} = f(S_{i-1}). \tag{5}$$

The nonzero Jacobian entries are

$$J_{i,i} = a + f'(S_i), \qquad J_{i,i-1} = -f'(S_{i-1}) \ (i \geq 1). \tag{6}$$

Column indices are stored in increasing order in a CRS triple (`ptr`, `col`, `val`). The Newton right-hand side is $-R$.

Each Newton iteration stops when $\|R\|_\infty < 10^{-9}$, and gives up after 15 iterations. The update is damped if a full step does not decrease the residual: the trial saturation is

$$S_i \leftarrow \mathrm{clamp}\big(S_i + \omega\,\delta_i,\ S_{wc},\ S_{\max}\big),$$

and ω starts at 1 and is halved until the infinity norm of R does not increase. Clamping keeps the relative permeabilities inside the Corey interval.

f and f' are analytic. `Fluid::flow` evaluates (6)'s derivative from the quotient rule on $\lambda_w(S)$ and $\lambda_o(S)$, including the factor ds/dS. A startup check compares that derivative with a centered difference on $(S_{wc}, S_{\max})$ and requires an error below 10^{-5}. The default data give about 1.7×10^{-10}.

4 Pressure: two-point flux

Mobility is frozen at the saturation just accepted, and k is the cell value (3). The half-transmissibility from a cell center to a face a distance $\Delta x/2$ away is

$$T_i^{\mathrm{half}} = \frac{2k_i\lambda_t(S_i)}{\Delta x}. \tag{7}$$

The face transmissibility is the harmonic mean of the two half-transmissibilities,

$$T_{i+1/2} = \frac{T_i^{\mathrm{half}}\,T_{i+1}^{\mathrm{half}}}{T_i^{\mathrm{half}} + T_{i+1}^{\mathrm{half}}}. \tag{8}$$

Finite-volume statements of (2), with the outlet face held at $p = 0$, are

$$\begin{aligned} T_{1/2}(p_0 - p_1) &= 1, \\ T_{i-1/2}(p_i - p_{i-1}) + T_{i+1/2}(p_i - p_{i+1}) &= 0, \\ T_{N-3/2}(p_{N-1} - p_{N-2}) + T^{\text{half}}_{N-1}\, p_{N-1} &= 0. \end{aligned} \tag{9}$$

The matrix is symmetric, with positive diagonal and nonpositive off-diagonal entries. The outlet Dirichlet condition removes the constant-pressure null space, so the matrix is symmetric positive definite and CG is appropriate. Because $k\lambda_t$ is cellwise constant, the exact solution of (9) is the recurrence

$$p_{N-1} = 1/T^{\text{half}}_{N-1}, \qquad p_i = p_{i+1} + 1/T_{i+1/2}, \tag{10}$$

which is also the integral of $dp = -dx/(k\lambda_t)$ from cell center to cell center. The AMGCL pressure is compared with (10), and every face flux $T(p_i - p_{i+1})$ is compared with 1. Both discrepancies are about 10^{-11} on the default run. This tests the linear solver, not a further modeling error: the discrete system is integrated exactly by (10).

5 Where AMGCL is called

AMGCL is a header-only algebraic multigrid library. The program uses the compile-time interface and defines `AMGCL_NO_BOOST`, so Boost is not required. The builtin backend stores the matrix in CRS. The adapter `amgcl/adapter/crs_tuple.hpp` accepts the system as

```
std::tie(n, ptr, col, val)
```

with `n` and the three arrays passed as non-const lvalues. The two solver types are

```
using Backend = amgcl::backend::builtin<double>;

using PressureSolver = amgcl::make_solver<
    amgcl::amg<Backend,
        amgcl::coarsening::smoothed_aggregation,
        amgcl::relaxation::spai0>,
    amgcl::solver::cg<Backend>>;

using TransportSolver = amgcl::make_solver<
    amgcl::amg<Backend,
        amgcl::coarsening::smoothed_aggregation,
        amgcl::relaxation::ilu0>,
    amgcl::solver::bicgstab<Backend>>;
```

`amgcl::make_solver` binds a preconditioner to an iterative method. The preconditioner is `amgcl::amg`: smoothed-aggregation coarsening, one pre-smoothing sweep and one post-smoothing sweep, and a direct skyline LU once the coarse problem has fewer than `coarse_enough` unknowns. That threshold is set to 60. The library default is large enough that an 800-unknown 1D matrix would otherwise collapse toward a single direct solve, and the printed hierarchy would not show several levels.

The pairing of method and smoother follows the matrix:

Unknown	Matrix	Iteration	AMG relaxation
pressure	SPD, tridiagonal	CG	SPAI(0)
saturation correction	nonsymmetric, bidiagonal	BiCGStab	ILU(0)

SPAI(0) is a diagonal sparse approximate inverse and preserves the SPD property that CG expects from the preconditioner. ILU(0) does not. It is a standard smoother for a nonsymmetric transport Jacobian, and on a bidiagonal pattern it is an exact factorization, so BiCGStab converges in one iteration. The V-cycle is still built. On the default mesh the mid-flood transport hierarchy has four levels and operator complexity about 1.70. The pressure hierarchy also has four levels, operator complexity about 1.48, and CG takes about 12.4 iterations on average. Relative residual tolerance is 10^{-10}, with a cap of 200 iterations. A solve whose reported residual exceeds 10^{-8} aborts the time step.

The hierarchy is reconstructed whenever the matrix values change. Saturation updates change $f'(S)$ and therefore J; they also change λ_t and therefore the pressure matrix. AMGCL can reuse transfer operators through `rebuild` when `allow_rebuild` is set. The program does not do that. Strength of connection in smoothed aggregation depends on the matrix entries, and the flood front moves those entries, so a fresh setup is the conservative choice on a problem this small.

A commercial fully implicit black-oil Jacobian is much larger: pressure and saturations (and often a well unknown) are coupled in one system. The usual preconditioner is CPR, constrained pressure residual. AMGCL provides it as `amgcl::preconditioner::cpr`. CPR still spends its setup on an AMG solve of a pressure block extracted from the full Jacobian; the global stage is an ILU sweep on the whole matrix. This program never forms that coupled matrix. The pressure AMG solve is written out on its own so that (10) can check it, and so that the saturation error can be checked against Welge.

6 Welge shock in the code

`Welge` maximizes $\sigma(S) = f(S)/(S - S_{wc})$ by golden-section search, then applies a scalar Newton correction to $g(S) = f'(S)(S - S_{wc}) - f(S)$. The shock speed stored for the CFL and for the exact profile is $v_s = f(S_f)/(S_f - S_{wc})$. The exact saturation at a cell is S_{wc} when $x_i/t \geq v_s$, and otherwise the bisection inverse of $f'(S) = x_i/t$ on $[S_f, S_{\max}]$. The numerical shock location reported in the text output is the first cell center where S drops below $\frac{1}{2}(S_f + S_{wc})$. Figure 1 plots that fractional flow. The hull is the chord from S_{wc} up to the point of tangency, and f itself beyond it.

7 Checks and a default run

The process returns success only if all of the following hold.

Check	Acceptance
$\|f' - f'_{\mathrm{fd}}\|_\infty$	$< 10^{-5}$
Welge tangent residual	$< 10^{-6}$
saturation L_1 against the Welge solution	< 0.03
$\|p - p_{\mathrm{exact}}\|_\infty$ from (10)	$< 10^{-6}$
face flux against 1	$< 10^{-6}$
conservation defect	$< 10^{-8}$

An 800-cell release run ($t = 0.241577$ pore volumes, $\Delta t = 5.49 \times 10^{-4}$, 440 steps) produced

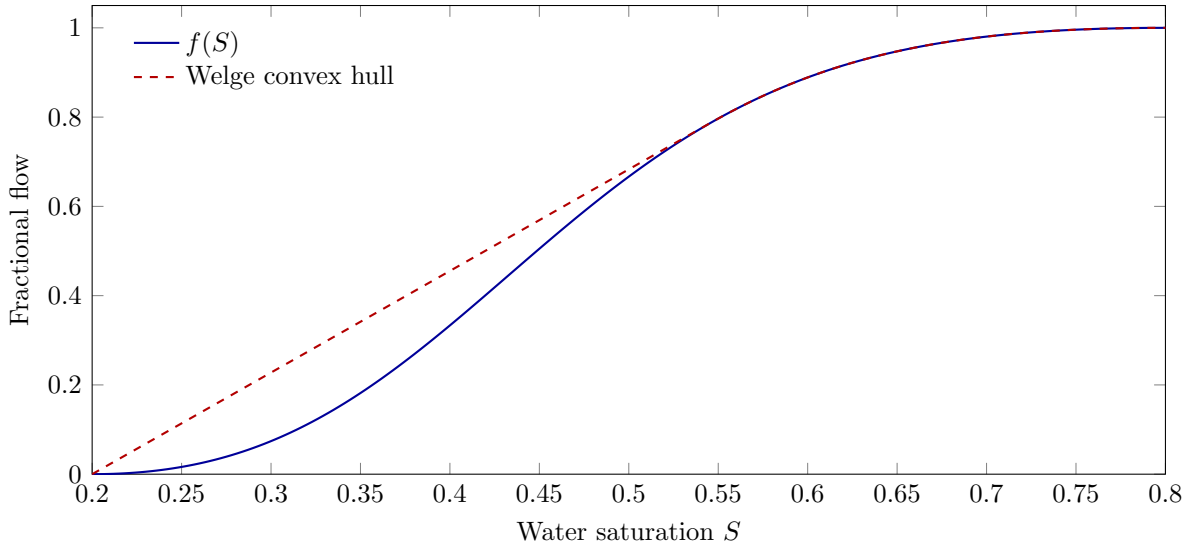


Figure 1: Fractional flow for the default Corey data, with the Welge convex hull. The hull is the tangent from S_{wc} up to $S_f = 0.54641$, then f itself.

Quantity	Value
S_f, v_s, breakthrough	0.54641, 2.27671, 0.43923
saturation L_1, L_∞	0.00414, 0.293
shock x, numerical / exact	0.558 / 0.550
pressure and face-flux errors	3.6×10^{-11}, 1.6×10^{-11}
conservation defect	1.7×10^{-15}
inlet pressure, $t = 0$ then t_{end}	3.799 then 4.008
pressure CG iterations, average	12.4
transport BiCGStab iterations, average	1

The L_∞ gap is the smeared shock: a first-order implicit upwind step cannot keep a discontinuity in one cell, while the Welge solution is sharp. The L_1 error is the meaningful accuracy measure, and it is a few thousandths of a saturation unit. The inlet pressure rises because $\lambda_t(S)$ for these Corey exponents has a minimum between S_{wc} and $S_{\max}$. The same rate then requires a larger drawdown. The tight streak adds a steep segment to $p(x)$ on $[0.45, 0.55]$, shaded in Figure 2. The exact discrete pressure lies on the AMGCL curve. The saturation plot is the same pair of curves the program writes to `output/profile.svg`.

8 Build and output

AMGCL 1.4.3 is the Git submodule `third_party/amgcl`. The repository records that release commit; `.gitmodules` only names the URL. Clone the project and that submodule together:

```
git clone --recurse-submodules ^
  https://github.com/konkolchin/buckley-leverett-amgcl.git
```

If the project is already present and `third_party/amgcl` is empty, fetch the recorded commit with

```
git submodule update --init
```

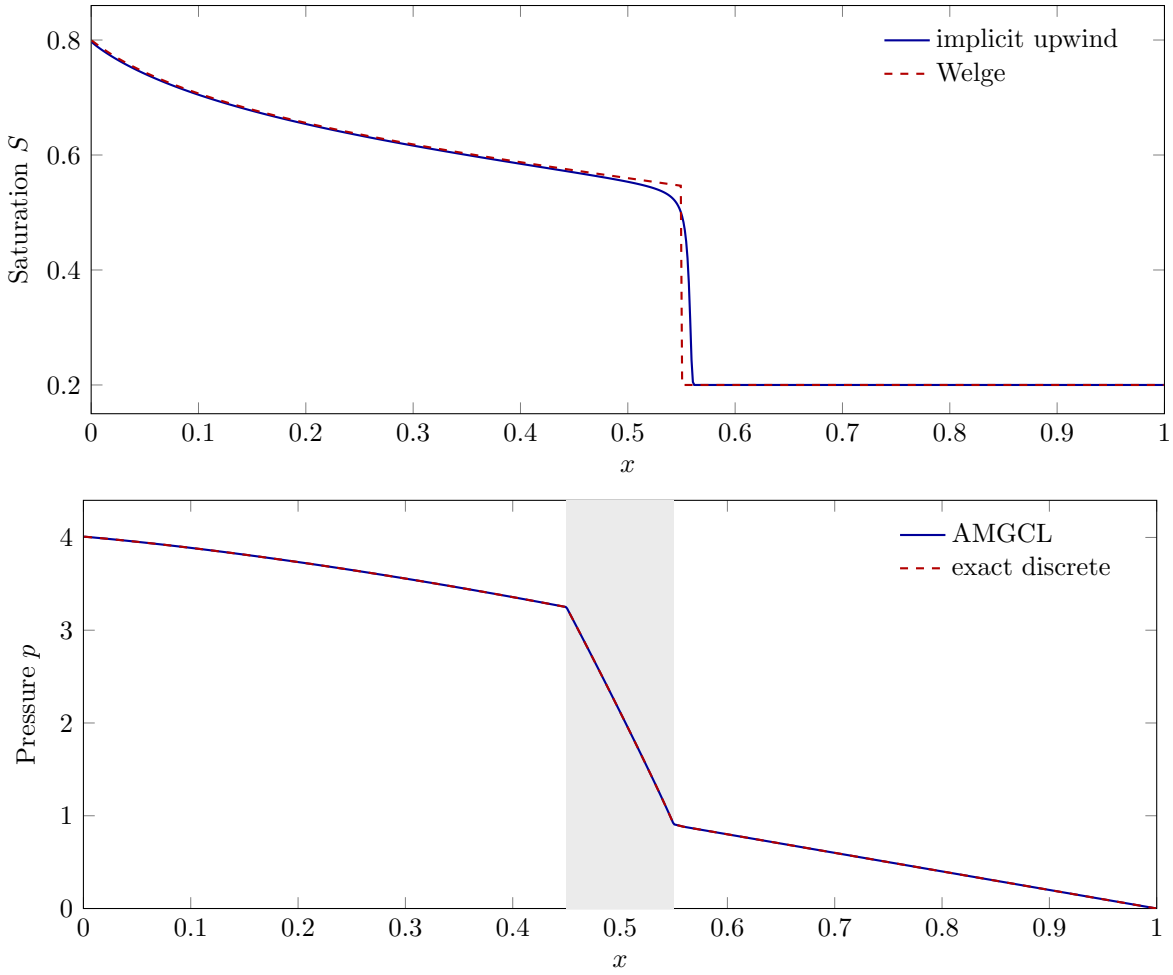


Figure 2: Default 800-cell run at $t = 0.242$ pore volumes, from `profile.csv`. Top: implicit upwind saturation and the Welge profile. The shock is spread over a few cells. Bottom: pressure. The shaded band is the streak $k = 0.1$.

From a Visual Studio 2022 x64 developer prompt, in the project directory,

```
cmake -S . -B build -G "Visual Studio 17 2022" -A x64
cmake --build build --config Release
build\Release\buckley.exe
```

The executable writes `output/`:

File	Contents
`report.txt`	AMG hierarchies, iteration counts, errors
`profile.csv`	x, S, Welge S, p, exact discrete p, k, λ_t, f
`fractional_flow.csv`	S, k_{rw}, k_{ro}, f, convex hull
`profile.svg`	saturation against Welge, and the pressure profile

The program is a study solver for the classical one-dimensional waterflood. The AMGCL calls are the real library interface: `make_solver`, smoothed aggregation, SPAI(0), ILU(0), CG, and

BiCGStab, with a CRS tuple as the matrix. It is not a multi-well black-oil simulator, and it does not exercise CPR.